

3F8: Inference

Short Lab Report

Yongqing Jiang

March 2, 2026

Abstract

Binary classification is the foundation supervised machine learning. This study investigates binary classification on a non-linearly separable dataset using Maximum Likelihood Estimation (MLE) to apply both linear and non-linear Gaussian Radial Basis Functions (RBF) classifier. While the linear model provides a limited accuracy, the RBF method significantly improves performance, achieving an 82% test accuracy through parameter selection. The results demonstrate that non-linear basis functions are essential for capturing complex decision boundaries and achieving better model generalization.

1 Introduction

In this lab activity, we design and implement a binary classifier using MLE to optimize the parameter β . The dataset presents a significant challenge due to its non-linearly separable distribution.

The performance of linear model and RBF method is rigorously assessed using the final average log-likelihood values and confusion matrices, providing insights into the trade-off between model complexity and generalization.

2 Exercise a)

In this exercise we have to consider the logistic classification model (aka logistic regression) and derive the gradients of the log-likelihood given a vector of binary labels $\mathbf{y}$ and a matrix of input features $\mathbf{X}$. The gradient of the log-likelihood can be written as

$$\frac{\partial \mathcal{L}(\beta)}{\partial \beta} = \sum_{n=1}^N (y^{(n)} - \sigma(\beta^T \tilde{x}^{(n)})) \tilde{x}^{(n)} = \tilde{\mathbf{X}}(\mathbf{y} - \sigma(\tilde{\mathbf{X}}\beta))$$

3 Exercise b)

In this exercise we are asked to write pseudocode to estimate the parameters β using gradient ascent of the log-likelihood. Our code should be vectorised. The pseudocode to estimate the parameters β is shown below:

Function `estimate_parameters`:

Input: feature matrix $\mathbf{X}$, labels $\mathbf{y}$
Output: vector of coefficients $\mathbf{b}$

Code:

```
for i from 0 to step_num begin
    sigmoid_val = sigmoid(X, beta)
```

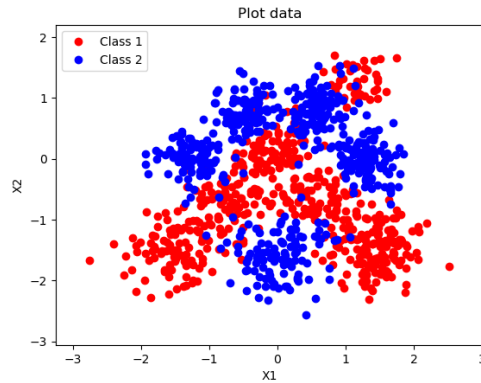


Figure 1: Visualisation of the data.

```

    b = b + alpha * (X(y - sigmoid_val))
end
return b

```

The learning rate parameter η is chosen to be 10^{-3} .

4 Exercise c)

In this exercise we visualise the dataset in the two-dimensional input space displaying each datapoint's class label. The dataset is visualised in Figure 1. By analysing Figure 1, we conclude that a linear classifier is not applicable on this problem. This distribution of class 1 and class 2 is clearly linearly inseparable as it has a non-linear class boundary.

5 Exercise d)

In this exercise we split the data randomly into training and test sets with 800 and 200 data points, respectively. The pseudocode from exercise a) is transformed into python code as follows:

```

def fit_w(X_tilde_train, y_train, X_tilde_test, y_test, n_steps, alpha):
    w = np.random.randn(X_tilde_train.shape[ 1 ])
    ll_train = np.zeros(n_steps)
    ll_test = np.zeros(n_steps)
    for i in range(n_steps):
        sigmoid_value = predict(X_tilde_train, w)
        w += alpha * (X_tilde_train.T@(y_train - sigmoid_value))
        ll_train[ i ] = compute_average_ll(X_tilde_train, y_train, w)
        ll_test[ i ] = compute_average_ll(X_tilde_test, y_test, w)
    return w, ll_train, ll_test

```

We then train the classifier using this code. We fixed the learning rate parameter to be $\eta = 10^{-3}$. The average log-likelihood on the training and test sets as the optimisation proceeds are shown in Figure 2. By looking at these plots we conclude that this model converges and it has a reasonable learning rate.

Figure 2 displays the visualisation of the contours of the class predictive probabilities on top of the data. This figure shows that our intuition about the limit of a linear classifier was correct. This classification can not be achieved by linear classifier. Non-linear terms should be used to classify.

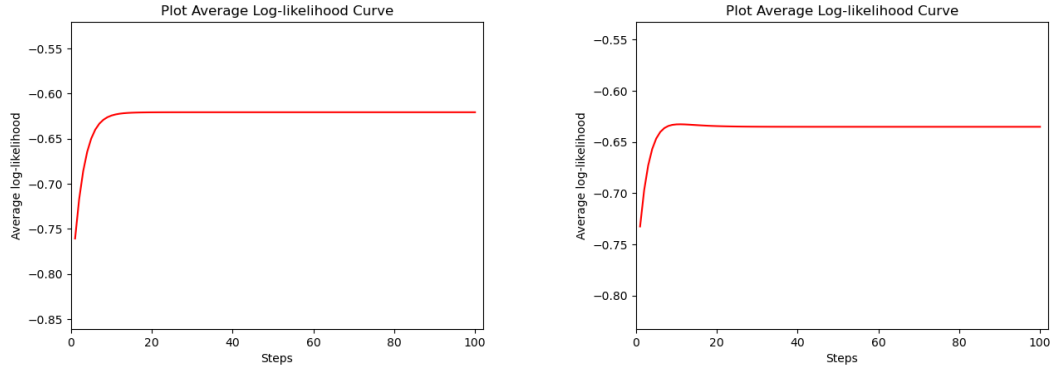


Figure 2: Learning curves showing the average log-likelihood on the training (left) and test (right) datasets.

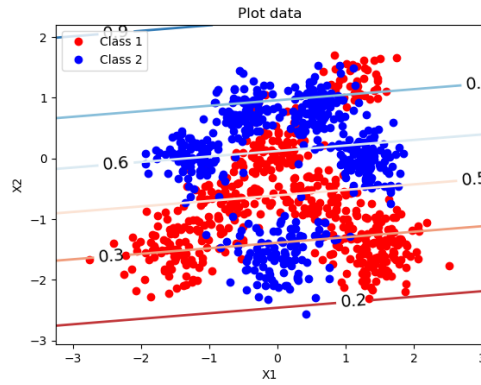


Figure 3: Visualisation of the contours of the class predictive probabilities.

Avg. Train ll	Avg. Test ll
-0.631	-0.596

y	$\hat{y}$	
	0	1
0	357	149
1	142	352

Table 1: Average training and test log-likelihoods.

Table 2: Confusion matrix on the test set.

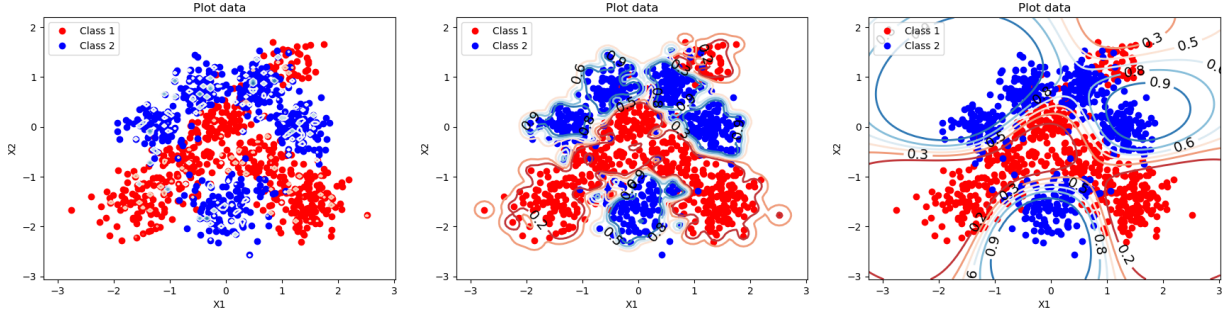


Figure 4: Visualisation of the contours of the class predictive probabilities for $l = 0.01$ (left), $l = 0.1$ (middle), $l = 1$ (right).

6 Exercise e)

The final average training and test log-likelihoods are shown in Table 1. These results indicate that this model underfits the data. These values around -0.6 implies the model does not classify. For reference, a random guess with 0.5 probabilities for each class yields a log-likelihood of $\ln(0.5) \approx -0.7$.

The 2x2 confusion matrices on the and test set is shown in Table 2. By analysing this table, we conclude that this model does not classify well. The accuracy is $\frac{357+352}{357+352+149+142} \approx 70\%$.

7 Exercise f)

We now expand the inputs through a set of Gaussian radial basis functions centred on the training datapoints. We consider widths $l = \{0.01, 0.1, 1\}$ for the basis functions. We fix the learning rate parameter to be $\eta = \{2 \times 10^{-3}, 5 \times 10^{-3}, 10^{-4}\}$ for each $l = \{0.01, 0.1, 1\}$, respectively. Figure 4 displays the visualisation of the contours of the resulting class predictive probabilities on top of the data for each choice of $l = \{0.01, 0.1, 1\}$.

8 Exercise g)

The final final training and test log-likelihoods per datapoint obtained for each setting of $l = \{0.01, 0.1, 1\}$ are shown in tables 3, 4 and 5. These results indicate that $l = 0.1$ provides the best classification among the values of l , since it has the highest value of log-likelihood. $e^{-0.197} = 0.821$ implies that it has a probability of 82% to classify correctly. $l = 1$ provides a slightly poorer classification. However, $l = 0.01$ lacks at generalization. The test set has a log-likelihood close to the theoretical minimum value $\ln(0.5) = -0.69$.

The 2×2 confusion matrices for the three models trained with $l = \{0.01, 0.1, 1\}$ are show in tables 6, 7 and 8. After analysing these matrices, we can say that $l = 0.1$ has the highest accuracy.

When we compare these results to those obtained using the original inputs we conclude that using Gaussian Radial Basis functions can obviously enhance the performance of the classifier. However, the parameters should be chosen with consideration, or it could also perform worse.

Avg. Train ll	Avg. Test ll	Avg. Train ll	Avg. Test ll	Avg. Train ll	Avg. Test ll
-0.390	-0.674	-0.120	-0.197	-0.259	-0.251

Table 3: Results for $l = 0.01$

Table 4: Results for $l = 0.1$

Table 5: Results for $l = 1$

		$\hat{y}$	
		0	1
y	0	476	30
	1	129	365

		$\hat{y}$	
		0	1
y	0	481	25
	1	31	463

		$\hat{y}$	
		0	1
y	0	451	55
	1	44	450

Table 6: Conf. matrix $l = 0.01$.

Table 7: Conf. matrix $l = 0.1$.

Table 8: Conf. matrix $l = 1$.

9 Conclusion

In this lab, we successfully designed a binary classifier using MLE. Our experiments confirm that while linear model fails to adequately model complex, intertwined datasets. Which is evidenced by a low average log-likelihood and significant misclassifications shown by the confusion matrix. By introducing Gaussian RBF, we achieved a substantial performance gain, increasing test accuracy to 82%. However, we also observed that hyperparameter sensitivity, mainly including learning rate and RBF bandwidth, plays a critical role in training stability. Future work could involve automated hyperparameter tuning to further refine the decision boundary.